the shortest path to a certain goal (either being a specific vertex, all vertices that can be reached from S, or a certain condition).

DFS, instead, is mostly used when we need to touch all the vertices, and we don’t care where we start. This algorithm provides great insight about a graph’s structure, and it’s used as a basis for several algorithms including finding a topological sorting for DAGs, or computing the strongly connected components of a directed graph.

The interesting bit about these algorithms is that their basic version, performing only the traversal, can be rewritten as the same templated algorithm, where newly discovered vertices are added to a container from which, at each iteration, we get the next element: for BFS, this container will be a queue and we’ll process vertices in the order they are discovered, while for DFS it will be a stack (implicit in the recursive version of this algorithm) and the traversal will try to go as far as possible, before backtracking and visit all vertex’s neighbors.

14.3.6 Best Route to Deliver a Parcel

Now that we have seen how BFS works and how to reconstruct the path from a source to a destination, we can go back to our example, and apply BFS to the graph in figure 14.9: the result is shown in figure 14.14.

In this case, the shortest path was pretty obvious, it’s the closest possible path to the “straight-line distance” between